

Universitatea Tehnică „Gheorghe Asachi” din Iași

Facultatea de Inginerie Electrică, Energetică și Informatică Aplicată

Proiect: „Sistem de Management și Analiză a Performanței Academice”,

Realizat: Mirăuța Vitalie, grupa 6112

Profesor: Monica Sălcianu

DOCUMENTAȚIE PROIECT

Sistem de Management și Analiză a Performanței Academice

1. Introducere și Motivația Proiectului

Proiectul constă într-o aplicație complexă pentru managementul și evaluarea performanței academice a studenților, fiind implementată în limbajul C și dotată cu o interfață text, precum și o componentă grafică de vizualizare.

Deși domeniul meu principal de studiu este energetica, dezvoltarea abilităților solide de programare (PCLP) este esențială în inginerie pentru automatizarea proceselor, prelucrarea semnalelor și analiza seturilor mari de date. Acest proiect mi-a permis să rezolv problema gestiunii manuale a informațiilor academice, scopul principal fiind automatizarea acestei gestiuni și furnizarea unor instrumente de analiză rapidă și eficientă a situației școlare. Sistemul oferă funcționalități complete de gestionare a datelor, înregistrare a rezultatelor și analiză vizuală.

2. Arhitectura și Structura Proiectului

Pentru a garanta un cod ușor de întreținut, am adoptat o arhitectură modulară. Sistemul este separat în două mari module:

1. **STUDENTLAUNCHER (Aplicația Core):** Reprezintă aplicația principală de consolă. Acesta gestionează interfața interactivă, logica de business (`student_logic.c`), structurile de date (`student_types.h`) și operațiile de tip I/O pentru stocarea datelor în fișiere (`student_io.c`).
2. **MEDII_OPENGL (Componenta Vizuală):** Un executabil separat, scris în C și OpenGL, care este apelat automat de aplicația principală pentru a randa diagrame cu mediile studenților și a permite comparații vizuale.

3. Structuri de Date Utilizate

Managementul corect al datelor este asigurat de structuri (struct-uri) bine definite, care folosesc alocarea dinamică a memoriei. Astfel, memoria este redimensionată automat doar la necesitate.

În student_types.h am definit entitățile principale:

- **SituatieStudent:** Reține istoricul academic folosind matrici alocate dinamic pentru note și credite.
- **DateStudent:** Agregă detaliile personale (nume, grupă, vârstă, un cod personal auto-incrementat) cu istoricul său academic.
- **StudentStorage:** Gestionarul principal care reprezintă o colecție dinamică de studenți.

Exemplu de definire a situației academice:

C

```
typedef struct {
    int semestre;

    int discipline;

    int **note;    // Matrice dinamică pentru note pe semestre
    int **credite; // Matrice dinamică pentru creditele alocate
    double *medii_ponderate;
    int *credite_semestru;
    int nr_discipline_peste_4_credite;
    int nr_restante_absente;
    int nr_semestre_scadere;
    int *semestre_scadere;
} SituatieStudent;
```

4. Logica de Calcul și Funcționalități

Sistemul pune la dispoziție un meniu interactiv complet, permițând adăugarea de studenți, introducerea situației școlare și generarea de clasamente și rapoarte.

O latură inginerească importantă a aplicației este procesarea algoritmică a indicatorilor de performanță. Media ponderată, esențială pentru evaluare, este calculată folosind formula clasică $\frac{\sum(\text{nota} \times \text{credit})}{\sum(\text{credite})}$.

Secvență de cod din student_logic.c (calculeaza_indicatori_scolari):

C

```
for (sem = 0; sem < s->semestre; sem++) {
```

```
suma_credite = 0;
suma_produs = 0;

for (d = 0; d < s->discipline; d++) {
    nota = s->note[sem][d];
    credit = s->credite[sem][d];

    if (nota == -1 || credit == -1) {
        continue; // Ignoră disciplinele neevaluate încă
    }

    suma_credite += credit;
    suma_produs += nota * credit;

    if (nota >= 0 && nota < 5) {
        s->nr_restante_absente++;
    }
}

s->credite_semestru[sem] = suma_credite;

// Calculul final al mediei ponderate per semestru
s->medii_ponderate[sem] = (suma_credite > 0) ? (double)suma_produs /
(double)suma_credite : 0.0;
}
```

5. Stocarea Datelor și Persistența

Fiind o aplicație administrativă, este vital ca datele să nu se piardă la închiderea programului. Sistemul de stocare folosește Standard C I/O (stdio.h) pentru a scrie și citi din fișiere. Stocarea datelor este organizată ierarhic pentru a reflecta structura unei facultăți, utilizând directoare pentru ani de studiu și grupe. Pentru a asigura integritatea

sistemului, am implementat o funcție care verifică și creează automat structura de foldere necesară folosind apeluri de sistem.

Logica de creare a directoarelor (student_io.c): Pentru portabilitate, am utilizat macro-ul MKDIR, adaptat în funcție de sistemul de operare (Windows/Linux).

C

```
int creeaza_director_daca_lipseste(const char *cale) {
    if (MKDIR(cale) == 0) { // Creează folderul (mkdir)
        return 1;
    }
    if (errno == EEXIST) { // Folderul există deja
        return 1;
    }
    return 0;
}
```

Persistența este garantată prin salvarea automată a fișierelor sub formatul [COD]_[NUME]_[PRENUME].txt în calea corespunzătoare (ex: StudentData/an_1/grupa_7/)

6. Componenta Grafică (OpenGL)

Aplicația depășește nivelul unei simple console. Modulul gui_launcher.c folosește API-ul Windows (ShellAPI.h) pentru a lansa procesul executabilului OpenGL creat separat, trimițându-i pe linia de comandă calea către fișierul studentului. Acest mod vizualizează mediile studenților și permite realizarea de comparații prin grafice interactive cu bare.

7. Vizualizarea Datelor: Componenta Grafică (OpenGL)

Din perspectiva unui viitor inginer, reprezentarea vizuală a datelor este crucială pentru identificarea tendințelor de performanță. Modulul grafic este un proces independent dezvoltat în OpenGL.

Lansarea modulului din aplicația principală (gui_launcher.c): Programul utilizează ShellExecuteA pentru a deschide componenta grafică, transmițând calea fișierelor ca parametri de lansare.

C

// Fragment lansare GUI cu parametri

```
rezultat_lansare = ShellExecuteA(NULL, "open", exe_grafic, parametri_lansare, NULL, SW_SHOWNORMAL);
```

```
if ((INT_PTR)rezultat_lansare <= 32) {  
    printf("Eroare la lansarea modulului grafic.\n");  
}
```

Randarea graficelor (medii_opengl.c): Am utilizat GLUT pentru gestionarea ferestrei și a evenimentelor de tastatură, permițând interacțiunea în timp real cu diagramele.

C

```
void display(void) {  
    glClear(GL_COLOR_BUFFER_BIT); // Curățare buffer  
    draw_axes(left, bottom, right, top); // Desenare axe coordonate  
    draw_bars(left, bottom, right, top); // Randare bare medii  
    glutSwapBuffers(); // Afișare cadru  
}
```

8. Sisteme de Automatizare și Optimizare

În ingineria electrică și energetică, eficiența unui sistem este dată de capacitatea sa de a se adapta la sarcini variabile și de a minimiza consumul de resurse. Proiectul integrează aceste concepte prin:

- **Optimizarea Memoriei (Alocare Dinamică):** Sistemul nu folosește vectori statici cu dimensiuni fixe. Funcția `asigura_capacitate_studenti` utilizează `realloc` pentru a dubla capacitatea de stocare doar atunci când este necesar, optimizând utilizarea RAM-ului.
- **Automatizarea Calculului Academic:** Toți indicatorii (medii ponderate, punctaje, credite) sunt calculați automat pe baza unor formule predefinite, eliminând eroarea umană.
- **Identificarea Automată a Riscurilor:** Programul scanează bazele de date și marchează automat „Indicatorii de risc”, cum ar fi restanțele (note sub 5) sau absențele excesive, funcționând ca un sistem de monitorizare a stării.

- **Sistem de Sortare și Clasament:** Implementarea unui algoritm de sortare pentru ierarhizarea studenților permite optimizarea procesului de selecție pentru burse sau alte distincții academice.

9. Concluzii și Posibilități de Extindere

Implementarea aplicației *Student Launcher* în limbajul C garantează o performanță ridicată și portabilitate, răspunzând eficient nevoilor de gestionare a datelor academice, raportare a performanței, persistență a informațiilor și analiză vizuală. Dincolo de simpla programare, lucrul cu pointeri, alocarea dinamică a memoriei și integrarea unor procese externe (OpenGL) mi-au format un mod de gândire analitic excelent pentru un viitor inginer.

Deși acum datele se stochează local (în fișiere text), un obiectiv pentru viitor este extinderea arhitecturii pentru a include o bază de date SQL dedicată și posibilitatea de export a datelor academice în formate uzuale precum Excel sau PDF.

10. Bibliografie și Resurse Utilizate

Realizarea acestui proiect a presupus o îmbinare între cunoștințele teoretice dobândite la facultate și cercetarea individuală necesară pentru rezolvarea unor probleme tehnice specifice (în special la interacțiunea cu sistemul de operare și componenta grafică).

10.1. Materiale Didactice (Curs și Laborator)

1. **Note de la Curs:** *Programarea Calculatoarelor și Limbaje de Programare (PCLP)*, Universitatea Tehnică „Gheorghe Asachi” din Iași, Facultatea de Inginerie Electrică, Energetică și Informatică Aplicată. (Referințe principale pentru gestionarea memoriei dinamice, structuri de date și lucrul cu fișiere).
2. **Laborator PCLP, Îndrumar de laborator:** *Limbajul C*, TUIASI. (Aprofundare practică pentru pointeri dubli utilizați în alocarea matricilor de note și credite).

10.2. Documentație Tehnică Oficială .

3. Microsoft Learn (Documentația API Windows). Utilizată pentru înțelegerea și implementarea funcțiilor dependente de sistemul de operare din modulul `gui_launcher.c` și `utils.c`:

- *ShellExecuteA function* - pentru lansarea executabilului OpenGL din aplicația principală.

- *GetModuleFileNameA* și *GetFileAttributesA* - pentru determinarea căilor absolute și verificarea existenței directoarelor.

4. OpenGL & FreeGLUT API Documentation. Referință pentru modulul *medii_opengl.c*, utilizată pentru setarea contextului vizual (*glutInit*, *glutCreateWindow*), randarea primitivelor geometrice (*glBegin(GL_QUADS)*) și gestionarea evenimentelor de la tastatură (*glutKeyboardFunc*).

10.3. Forumuri și Comunități de Dezvoltatori (Adaptare funcții complexe) .

5. Stack Overflow: „*How to recursively create directories in C on Windows/Linux?*” *

Utilitate în proiect: Această discuție a servit ca inspirație pentru implementarea funcției *creaza_director_daca_lipseste()* din *student_io.c*. Abordarea de a utiliza macro-ul *MKDIR* și de a verifica condiția de eroare *errno == EEXIST* (când folderul există deja) este o practică standard preluată din comunitate pentru a asigura compatibilitatea cross-platform.

6. Stack Overflow / C-Board: „*Getting the path of the current executable in C*”

- **Utilitate în proiect:** Funcția *obține_cale_executabil()* din *utils.c*, care trunchiază calea la ultimul separator \ pentru a găsi folderul rădăcină (evitând folderele *Debug* sau *Release*), a fost adaptată pornind de la soluții discutate pe aceste forumuri, fiind esențială pentru portabilitatea proiectului pe alte calculatoare.

7. Forumuri de Game Development (OpenGL): „*Drawing bar charts and scaling axes in raw OpenGL*”

- **Utilitate în proiect:** Conceptul de calcul matematic al lățimii barelor (*bar_w*) în funcție de dimensiunea ferestrei dinamice (*resize*) și implementarea modului de comparare cu decalaje (*group_gap*) în funcția *drawBarsCompare()* au fost perfecționate studiind exemple de grafică 2D de pe forumuri de specialitate.